PHASE 1 – REQUIREMENT ANALYSIS & PROJECT PLANNING

Case 09: Railway Reservation System

Process Model: V-Model | **Agile Methodology:** Scrum | **AI Tool:** ChatGPT

This document covers all 10 steps of Phase 1. It establishes the requirements, project scope, stakeholders, feasibility, SRS, V-Model approach, Scrum backlog and user stories.

The assigned scenario and prototype from your notebook will remain unchanged.

STEP 1 – PROBLEM UNDERSTANDING

1.1 Project Background

Railway reservation systems experience extremely high traffic during the Tatkal booking period. When booking opens at 10:00 AM, thousands of passengers attempt to reserve seats simultaneously.

When multiple users request the same available seat, race conditions may occur. If the system does not implement proper concurrency control, two users could potentially receive confirmation for the same seat.

The proposed Railway Reservation System focuses on developing a reliable seat allocation mechanism that prevents duplicate bookings through concurrency control.

1.2 Problem Statement

The Railway Reservation System must handle a high volume of concurrent booking requests during the Tatkal reservation period. Up to 50,000 users may attempt to book tickets within one minute. The system must ensure that each seat is allocated to only one passenger, even when multiple users simultaneously request the same seat.

The project aims to develop a prototype using concurrency locking mechanisms to prevent race conditions, maintain database consistency and provide reliable booking confirmation.

1.3 Project Objectives

1. Develop a working railway seat reservation prototype.
2. Display available and booked seats.
3. Allow passengers to select and reserve seats.
4. Implement concurrency control during seat allocation.
5. Prevent duplicate seat allocation.
6. Maintain consistent booking records.
7. Provide booking confirmation.
8. Evaluate system performance under concurrent requests.
9. Maintain software quality through automated testing and static analysis.

1.4 Major Challenges

Challenge	Description
High Traffic	50,000 booking attempts within one minute.
Race Conditions	Multiple users requesting the same seat simultaneously.
Double Booking	More than one passenger receiving the same seat.
Database Consistency	Maintaining correct seat and booking information.
Performance	Achieving the 2-second response-time target.
Reliability	Maintaining correct operation during peak demand.

1.5 Expected Outcome

A working prototype demonstrating concurrent seat allocation, prevention of double booking, booking confirmation and reliable database updates.

STEP 2 – PROJECT SCOPE DEFINITION

2.1 Project Title

Railway Reservation System – Concurrent Seat Allocation Simulator

2.2 In-Scope Features

Feature ID	Feature	Description
F01	Seat Availability	Display available and booked seats.
F02	Seat Selection	Allow passengers to select seats.
F03	Seat Booking	Process reservation requests.
F04	Concurrency Control	Handle simultaneous booking requests.
F05	Double Booking Prevention	Ensure a seat is allocated only once.
F06	Booking Confirmation	Display successful booking details.
F07	Booking History	Display previous booking records.

2.3 Out-of-Scope Features

The following features will not be implemented in the prototype:

- Online payment gateway.
- Train scheduling and route management.
- Ticket cancellation and refunds.
- PNR generation and tracking.
- Integration with actual railway infrastructure.
- Complete IRCTC reservation functionality.

2.4 Project Boundaries

The project will concentrate on the seat allocation and concurrency control module.

Other railway reservation features may be discussed conceptually but are not part of the working prototype.

2.5 Project Constraints

- Booking opening time: 10:00 AM.
- Maximum scenario load: 50,000 booking attempts per minute.
- Double booking is unacceptable.
- The assigned process model must be V-Model.
- The assigned Agile method must be Scrum.
- ChatGPT must be used as the assigned AI tool.

STEP 3 – STAKEHOLDER IDENTIFICATION

3.1 Stakeholder Analysis

Stakeholders are individuals or groups who interact with, influence or are affected by the system.

Stakeholder	Role	Responsibility
Passenger	Primary User	Views and books available seats.
Administrator	System Manager	Monitors bookings and seat allocation.
Development Team	Developers	Designs and implements the application.
Testing Team	Testers	Tests functionality and concurrency.
Project Owner	Business Stakeholder	Defines requirements and expectations.
Database System	Technical Component	Stores booking and seat information.

3.2 Stakeholder Requirements

Stakeholder	Requirement
Passenger	Easy seat selection and booking confirmation.
Administrator	Accurate booking information.
Developer	Clear requirements and maintainable architecture.
Tester	Testable requirements and expected results.
Project Owner	Reliable booking without duplicate allocations.

3.3 Primary and Secondary Actors

Primary Actor: Passenger.

Secondary Actors: Administrator and database system.

3.4 Stakeholder Expectations

The system should provide:

- Accurate seat availability.
- Reliable reservation processing.
- Consistent booking records.
- Fast booking responses.
- Prevention of duplicate seat allocation.

STEP 4 – FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that the Railway Reservation System must perform.

4.1 Functional Requirements Specification

Requirement ID	Requirement	Description	Priority
FR-01	Display Seats	The system shall display available and booked seats.	High
FR-02	Select Seat	The system shall allow passengers to select available seats.	High
FR-03	Booking Request	The system shall accept booking requests from multiple users.	High
FR-04	Availability Check	The system shall verify seat availability before allocation.	Critical
FR-05	Concurrency Control	The system shall implement concurrency control during seat allocation.	Critical
FR-06	Prevent Double Booking	The system shall prevent multiple passengers from booking the same seat for the same journey.	Critical
FR-07	Update Seat Status	The system shall update seat availability after successful booking.	High
FR-08	Booking Confirmation	The system shall display booking confirmation details.	High
FR-09	Reject Duplicate Requests	The system shall reject requests for already-booked seats.	High

FR-10	Store Booking Records	The system shall maintain booking information in the database.	Medium

4.2 Functional Requirement Example

FR-06: Prevent Double Booking

Description:

The system must ensure that when multiple users attempt to reserve the same seat simultaneously, only one booking request is successful.

Expected behaviour:

User	Request	Result
User A	Book Seat 45	Successful
User B	Book Seat 45	Rejected
User C	Book Seat 45	Rejected

Only one confirmed booking should exist for Seat 45 on that journey.

STEP 5 – NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the performance, reliability, security and quality expectations of the system.

5.1 Non-Functional Requirements Specification

Requirement ID	Category	Requirement	Measurement
NFR-01	Performance	95% of bookings should complete within 2 seconds.	95th percentile response time

NFR-02	Scalability	System should be evaluated against 50,000 booking attempts per minute.	Load testing
NFR-03	Reliability	The system must prevent double booking.	Duplicate booking count = 0
NFR-04	Consistency	Booking and seat records must remain consistent.	Database validation
NFR-05	Security	Booking information must be protected against unauthorised access.	Security testing
NFR-06	Maintainability	The application must have modular and readable code.	Code quality analysis
NFR-07	Availability	The booking service should remain operational during peak demand.	Availability monitoring
NFR-08	Usability	Users should be able to book seats through a simple interface.	User acceptance testing

5.2 Performance Target

The assigned scenario contains two important performance parameters:

- 50,000 booking attempts within one minute.
- 95% of bookings under 2 seconds.

These will be used as performance-testing targets.

Important: These are system targets, not demonstrated prototype capabilities. Actual capacity must be measured. A prototype using Flask and SQLite may require a different architecture to achieve production-scale performance.

STEP 6 – FEASIBILITY AND CONSTRAINTS ANALYSIS

6.1 Technical Feasibility

The project is technically feasible as a prototype using commonly available technologies.

Component	Proposed Technology
Programming Language	Python
Frontend	HTML, CSS, JavaScript
Backend	Flask
Database	SQLite
Concurrency Control	Threading Lock and Database Transactions
Testing	PyTest and Selenium
Version Control	Git and GitHub
CI/CD	Jenkins
Containerization	Docker
Code Quality	SonarQube

A database transaction and appropriate locking mechanism will be used to ensure that concurrent requests cannot allocate the same seat.

For a production multi-process or distributed deployment, database-level concurrency control is essential; a process-local threading lock alone is insufficient.

6.2 Operational Feasibility

The application will provide a simple interface where passengers can:

1. View available seats.
2. Select a seat.
3. Submit a booking request.
4. Receive booking confirmation or rejection.

The system should be easy to understand and operate.

6.3 Economic Feasibility

The prototype can be developed using open-source technologies.

Technology	Cost

Python	Free
Flask	Free
SQLite	Free
GitHub	Free tier available
Selenium	Free
Docker	Free tier available
Jenkins	Open source
SonarQube Community Build	Free

6.4 Project Constraints

1. Limited prototype scope.
2. High concurrent booking demand.
3. Strict prevention of double booking.
4. Assigned V-Model process model.
5. Assigned Scrum methodology.
6. ChatGPT as the specified AI tool.
7. Five sprints with a total planned workload of 80 story points.

6.5 Feasibility Conclusion

The proposed prototype is feasible using Python, Flask and database transactions. However, achieving the complete 50,000-user scenario requires dedicated performance engineering and infrastructure beyond the basic prototype.

STEP 7 – SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

7.1 Introduction

The Software Requirements Specification defines the functional and non-functional requirements of the Railway Reservation System.

It acts as the primary reference for system design, development, testing and maintenance.

7.2 Purpose

The purpose of this project is to develop a seat allocation prototype that can process concurrent booking requests while preventing duplicate seat allocation.

7.3 Scope

The system focuses on seat availability, seat selection, booking processing, concurrency control and booking confirmation.

7.4 Overall System Description

The application consists of the following components:

1. User Interface.
2. Booking Management Module.
3. Seat Allocation Module.
4. Concurrency Control Module.
5. Database Management Module.
6. Booking Confirmation Module.

7.5 User Classes

User	Description
Passenger	Books available seats.
Administrator	Monitors bookings.

7.6 Functional Requirements

The system must satisfy FR-01 through FR-10 defined in Step 4.

7.7 Non-Functional Requirements

The system must satisfy NFR-01 through NFR-08 defined in Step 5.

7.8 External Interface Requirements

User Interface:

- Seat availability dashboard.

- Seat selection interface.
- Booking confirmation screen.
- Booking history screen.

Software Interface:

- Flask backend.
- SQLite database.
- Browser-based frontend.

7.9 Database Requirements

The prototype will initially use three primary entities.

Users Table

Attribute	Description
user_id	Unique user identifier
username	User name
email	User email

Seats Table

Attribute	Description
seat_id	Unique seat identifier
journey_id	Journey identifier
status	Available or booked

Bookings Table

Attribute	Description
booking_id	Unique booking identifier
user_id	Passenger identifier
seat_id	Selected seat
journey_id	Journey identifier

booking_status	Confirmed or rejected
booking_time	Booking timestamp

A database uniqueness constraint will prevent multiple confirmed bookings for the same seat and journey.

7.10 System Constraints

- The system must prevent double booking.
- Booking requests must be handled safely.
- Seat status must remain consistent.
- The system must be testable using automated test cases.

7.11 Acceptance Criteria

The prototype will be considered functionally acceptable when:

1. Available seats are displayed correctly.
2. Users can submit booking requests.
3. Only one user can book a particular seat for a journey.
4. Concurrent requests are handled safely.
5. Booking records are stored correctly.
6. Confirmation or rejection messages are displayed.

STEP 8 – V-MODEL PLANNING

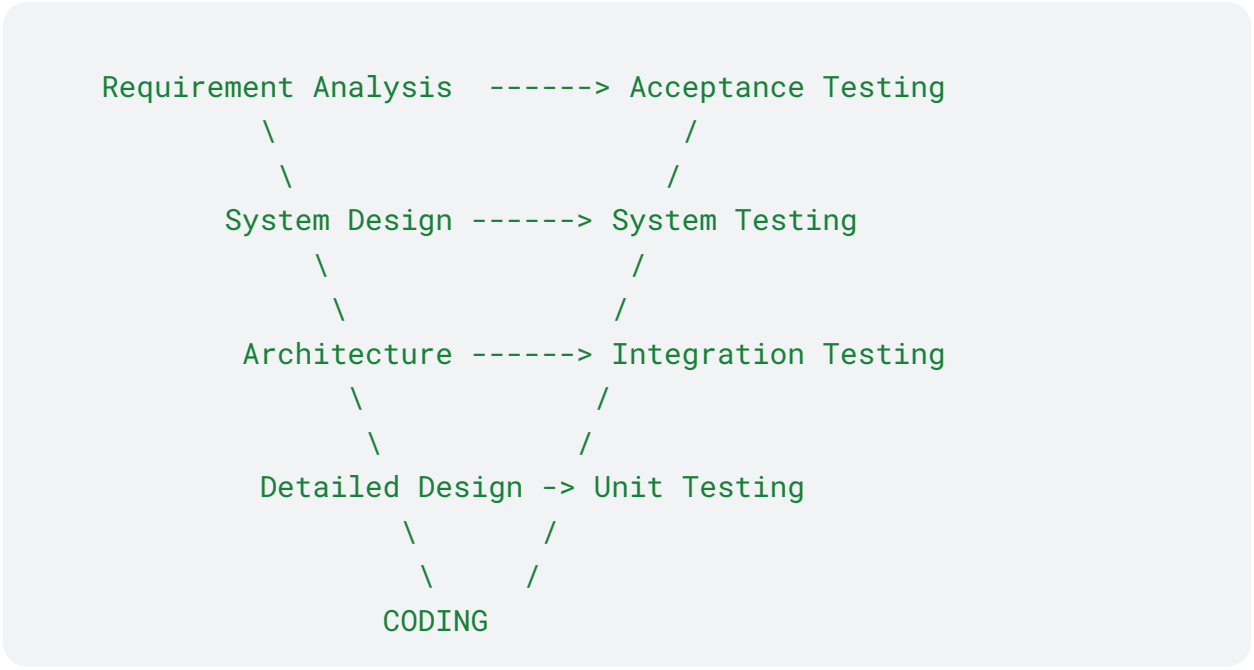
8.1 Introduction

The V-Model is a software development lifecycle model in which each development phase has a corresponding testing phase.

It emphasises verification and validation throughout development.

8.2 V-Model Diagram

```
None
        VERIFICATION                    VALIDATION
```



8.3 Development and Testing Mapping

Development Phase	Corresponding Testing
Requirement Analysis	Acceptance Testing
System Design	System Testing
Architecture Design	Integration Testing
Detailed Design	Unit Testing
Coding	Implementation

8.4 Application to Railway Reservation System

Requirement Analysis:

Identify the need for concurrent seat booking and double-booking prevention.

Corresponding test: Acceptance testing verifies that passengers receive correct booking results.

System Design:

Design the complete application architecture.

Corresponding test: System testing verifies the complete booking workflow.

Architecture Design:

Define interactions between frontend, backend and database.

Corresponding test: Integration testing checks communication between components.

Detailed Design:

Design the seat allocation algorithm and concurrency mechanism.

Corresponding test: Unit testing verifies the allocation function and locking logic.

Coding:

Implement the application.

8.5 Why V-Model Fits This Project

1. Double booking is a critical failure that requires systematic verification.
2. Testing can be planned alongside requirements and design.
3. Requirements can be traced to corresponding test cases.
4. Performance and acceptance criteria can be defined early.

8.6 When Another Model Might Be Needed

An Incremental or Agile-oriented development approach could be more suitable if requirements change frequently or the system needs continuous feature releases.

However, the assigned process model for this case remains V-Model.

STEP 9 – SCRUM PRODUCT BACKLOG

9.1 Introduction

Scrum is an Agile framework that divides development work into short iterations called sprints.

Your assignment specifies:

Parameter	Assigned Value
Team Members	3

Team Distribution	Distributed
Velocity	18 story points
Total Planned Work	80 story points
Number of Sprints	5
AI Tool	ChatGPT

9.2 Scrum Roles

Role	Responsibility
Product Owner	Prioritises requirements and manages the backlog.
Scrum Master	Facilitates Scrum events and removes blockers.
Developer	Develops and tests the application.

All three members should participate in development and testing, even if responsibilities are distributed.

9.3 Product Backlog

Epic ID	Epic Name	Description	Priority
EP-01	Seat Availability	Display available and booked seats.	High
EP-02	Seat Selection	Allow passengers to select seats.	High
EP-03	Seat Allocation	Allocate seats to passengers.	Critical
EP-04	Concurrency Control	Handle simultaneous booking requests.	Critical
EP-05	Booking Confirmation	Generate booking confirmation.	High
EP-06	Testing and Quality	Verify system functionality and performance.	High
EP-07	Deployment	Automate build and deployment.	Medium

9.4 Five-Sprint Plan

Sprint	Main Activities	Planned Story Points

Sprint 1	Requirements and initial design	16
Sprint 2	Architecture, database and detailed design	16
Sprint 3	Seat allocation and concurrency implementation	18
Sprint 4	Integration and automated testing	18
Sprint 5	Performance testing, Jenkins, Docker and quality assurance	12
Total		**80**

The planned workload is 80 points against a nominal five-sprint capacity of 90 points.

9.5 Scrum Events

Event	Frequency	Purpose
Sprint Planning	Beginning of sprint	Select sprint work.
Daily Scrum	Daily	Discuss progress and blockers.
Sprint Review	End of sprint	Demonstrate completed work.
Sprint Retrospective	End of sprint	Identify improvements.

9.6 Scrum Artifacts

- Product Backlog.
- Sprint Backlog.
- Product Increment.

9.7 AI-Assisted Planning

ChatGPT can help:

- Estimate story points.
- Identify dependencies.
- Summarise backlog items.
- Suggest sprint allocations.
- Identify potentially overloaded tasks.

AI estimates must be compared against manual team estimates.

STEP 10 – USER STORIES AND ACCEPTANCE CRITERIA

User stories describe system requirements from the perspective of the end user.

10.1 User Stories

User Story ID	User Story	Requirement Mapping	Priority
US-01	As a passenger, I want to view available seats so that I can choose a seat.	FR-01	High
US-02	As a passenger, I want to select a seat so that I can request a reservation.	FR-02	High
US-03	As a passenger, I want to submit a booking request so that I can reserve a seat.	FR-03	High
US-04	As a passenger, I want the system to prevent duplicate bookings so that my reservation remains unique.	FR-04, FR-05, FR-06	Critical
US-05	As a passenger, I want to receive booking confirmation so that I know whether my reservation succeeded.	FR-08	High
US-06	As a passenger, I want unavailable seats to be rejected so that incorrect bookings cannot occur.	FR-09	High
US-07	As an administrator, I want to view booking records so that I can monitor reservations.	FR-10	Medium
US-08	As an administrator, I want to monitor system performance so that I can identify booking failures.	NFR-01, NFR-02	High

10.2 Acceptance Criteria

US-01: View Available Seats

- Given the passenger opens the booking interface,
- When the system loads seat information,
- Then all available and booked seats should be displayed correctly.

US-02: Select a Seat

- Given a seat is available,
- When a passenger selects it,
- Then the system should accept the selection.

US-04: Prevent Double Booking

- Given only one seat is available,
- When two passengers simultaneously request that seat,
- Then only one booking should succeed and the other should be rejected.

US-05: Booking Confirmation

- Given a booking request is successfully processed,
- When the database confirms the allocation,
- Then the system should display booking confirmation.

US-06: Reject Unavailable Seats

- Given a seat is already booked,
- When another passenger requests that seat,
- Then the system should reject the request.

10.3 Definition of Done

A user story will be considered complete when:

1. The required functionality is implemented.
2. The code passes relevant unit tests.
3. Integration tests pass where applicable.
4. Code review is completed.
5. No duplicate seat allocation is possible.
6. Documentation is updated.
7. Changes are committed to GitHub.

PHASE 1 – FINAL SUMMARY

All ten steps have now been prepared.

Step	Deliverable	Status
1	Problem Understanding	Completed

2	Project Scope Definition	Completed
3	Stakeholder Identification	Completed
4	Functional Requirements	Completed
5	Non-Functional Requirements	Completed
6	Feasibility and Constraints	Completed
7	SRS Documentation	Initial version completed
8	V-Model Planning	Completed
9	Scrum Product Backlog	Initial version completed
10	User Stories and Acceptance Criteria	Completed

Important: Assignment Coverage

Your notebook also requires five specific case tasks:

Task	Description	Phase Connection
Task A	V-Model and 10 load scenarios	Steps 1, 5, 8
Task B	Scrum planning and AI/manual forecasts	Steps 9, 10
Task C	AI coding, race condition, Git, Jenkins and Docker	Future implementation phase
Task D	Concurrency testing, Selenium and monitoring	Future testing phase
Task E	Performance target, SonarQube and fairness	Future quality phase

The remaining 25 theory questions and 25 lab exercises must also be completed separately using their original numbering.

Phase 1 Completion Decision

The planning foundation is ready. However, the SRS, V-Model and backlog are initial versions and should be refined as we progress through the assignment.

Next: PHASE 2 – SYSTEM DESIGN

We will work on:

1. System architecture.
2. Technology stack finalisation.
3. Database design and ER diagram.
4. UML use-case diagram.
5. UML class diagram.
6. Component and data-flow design.

We will preserve the same FR, US and TC identifiers throughout the project.